ANÁLISIS DEL RETO

Juan Sebastian Rojas Marin, js.rojasm123@uniandes.edu.co, 202610140

Katy Teran Mercado, k.teranm@uniandes.edu.co, 202614247

Juan Jose Rincon Santos, jj.rincons1@uniandes.edu.co, 202615789

Requerimiento <<1>>

Descripción



```
def req_1(catalog, anio, mes, descuento_min, descuento_max):
    def criterio(pedido_1, pedido_2):
        return pedido_1["Discount_Pct"] > pedido_2["Discount_Pct"]
    if pedido_1["Amount"] != pedido_2["Amount"]:
        return pedido_1["Amount"] > pedido_2["Amount"]
    return pedido_1["Order_ID"] <= pedido_2["Order_ID"]

start_time = get_time()

llave_mes = str(int(anio)).zfill(4) + "-" + str(int(mes)).zfill(2)
pedidos_mes = mlp.get(catalog["OrdenMes"], llave_mes)

filtrados = lt.new_list()
total_amount = 0.0
suma_precio = 0.0
suma_cajas = 0

if pedidos_mes is not None:
    for i in range(lt.size(pedidos_mes)):
        pedido = lt.get_element(pedidos_mes, i)
        descuento = pedido["Discount_Pct"]
        if descuento >= descuento_min and descuento <= descuento_max:
            lt.add_last(filtrados, pedido)
            total_amount += pedido["Amount"]
            suma_precio += pedido["Price_per_Box"]
            suma_cajas += pedido["Boxes_Shipped"]

total_pedidos = lt.size(filtrados)
promedio_precio = 0.0
promedio_cajas = 0.0
if total_pedidos > 0:
    lt.merge_sort(filtrados, criterio)
    promedio_precio = suma_precio / total_pedidos
    promedio_cajas = suma_cajas / total_pedidos
```

```
    end_time = get_time()
    return (delta_time(start_time, end_time), total_pedidos, total_amount,
            promedio_precio, promedio_cajas, filtrados)
```

Consulta los pedidos de un mes específico cuyo descuento esté dentro de un rango dado, sin recorrer la lista principal. Durante la carga se construye la tabla de hash OrdenMes (Linear Probing), con llave "YYYY-MM" y valor un array_list con los pedidos de ese mes. Con el año y el mes recibidos se reconstruye la llave y se accede al mes en una sola operación. Sobre ese subconjunto se filtra por descuento acumulando en el mismo recorrido el monto total, la suma de precios y la suma de cajas, para no hacer pasadas adicionales. Los pedidos que cumplen se guardan en un array_list que se ordena con Merge Sort. Si el mes no existe, mlp.get retorna None y el resultado es vacío sin error.

Entrada	catalog, anio, mes, descuento min, descuento_max
Salidas	tiempo, total pedidos, total_amount, promedio precio, promedio cajas, filtrados.
Implementado (Sí/No)	Implementado por Juan Rojas

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	**Complejidad**
Construir la llave "YYYY-MM" a partir del año y el mes	O(1)
Buscar la llave en la tabla de hash OrdenMes (mlp.get, Linear Probing)	O(1) promedio
Recorrer los m pedidos del mes y filtrar por el rango de descuento, acumulando monto total, suma de precios y suma de cajas	O(m)
Ordenar los k pedidos filtrados con Merge Sort	O(k log k)
Calcular los dos promedios a partir de los acumuladores	O(1)
TOTAL	***O(m + k log k)***

Análisis

Indexar por mes evita recorrer los 193.608 pedidos: cada mes hay aproximadamente 8.000 registros y la búsqueda es O(1), así que el costo depende del tamaño del mes y del resultado, no del archivo. Se usó Linear Probing porque solo hay 24 llaves (los meses de 2022 y 2023) sobre capacidad 211, con factor de carga real de 0.11 y colisiones casi nulas; Separate Chaining añadiría una lista encadenada por casilla sin necesidad. El ordenamiento es Merge Sort, O(k log k) garantizado y estable.